An IR for SVF

Mohamad Barbar

October 2026

As it stands now...

1. Feed C/C++ into Clang to produce LLVM IR
2. Feed LLVM IR to SVF to produce graphs
3. Analyse graphs

As it stands now...

1. Feed C/C++ into Clang to produce LLVM IR
2. Feed LLVM IR to SVF to produce graphs
3. Analyse graphs

Other languages?

As it stands now...

1. Feed C/C++ into Clang to produce LLVM IR
2. Feed LLVM IR to SVF to produce graphs
3. Analyse graphs

Other languages?

- If they have LLVM producing compilers (and you're lucky)

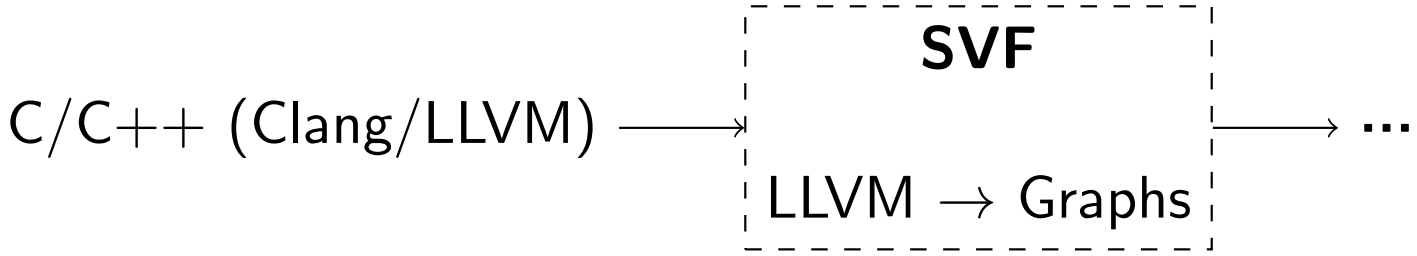
As it stands now...

1. Feed C/C++ into Clang to produce LLVM IR
2. Feed LLVM IR to SVF to produce graphs
3. Analyse graphs

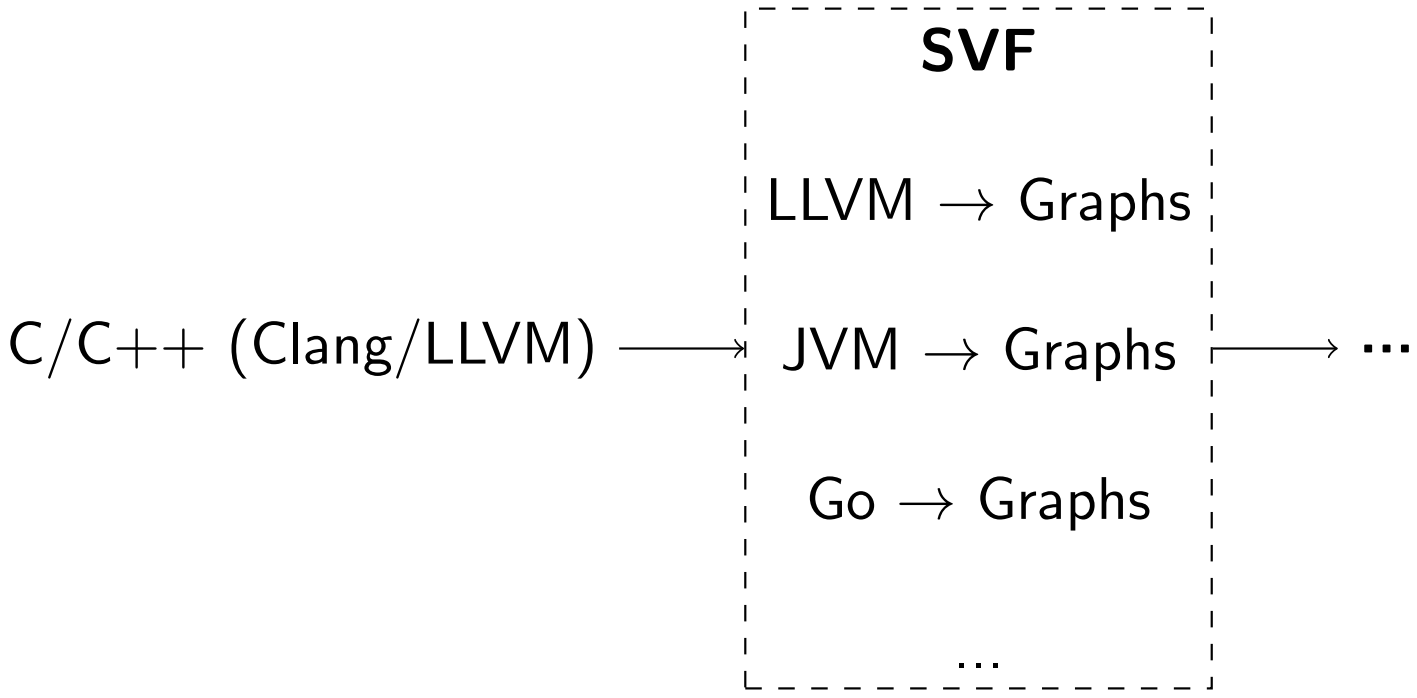
Other languages?

- If they have LLVM producing compilers (and you're lucky)
- If they have a translator to LLVM (and you're lucky)

As it stands now...



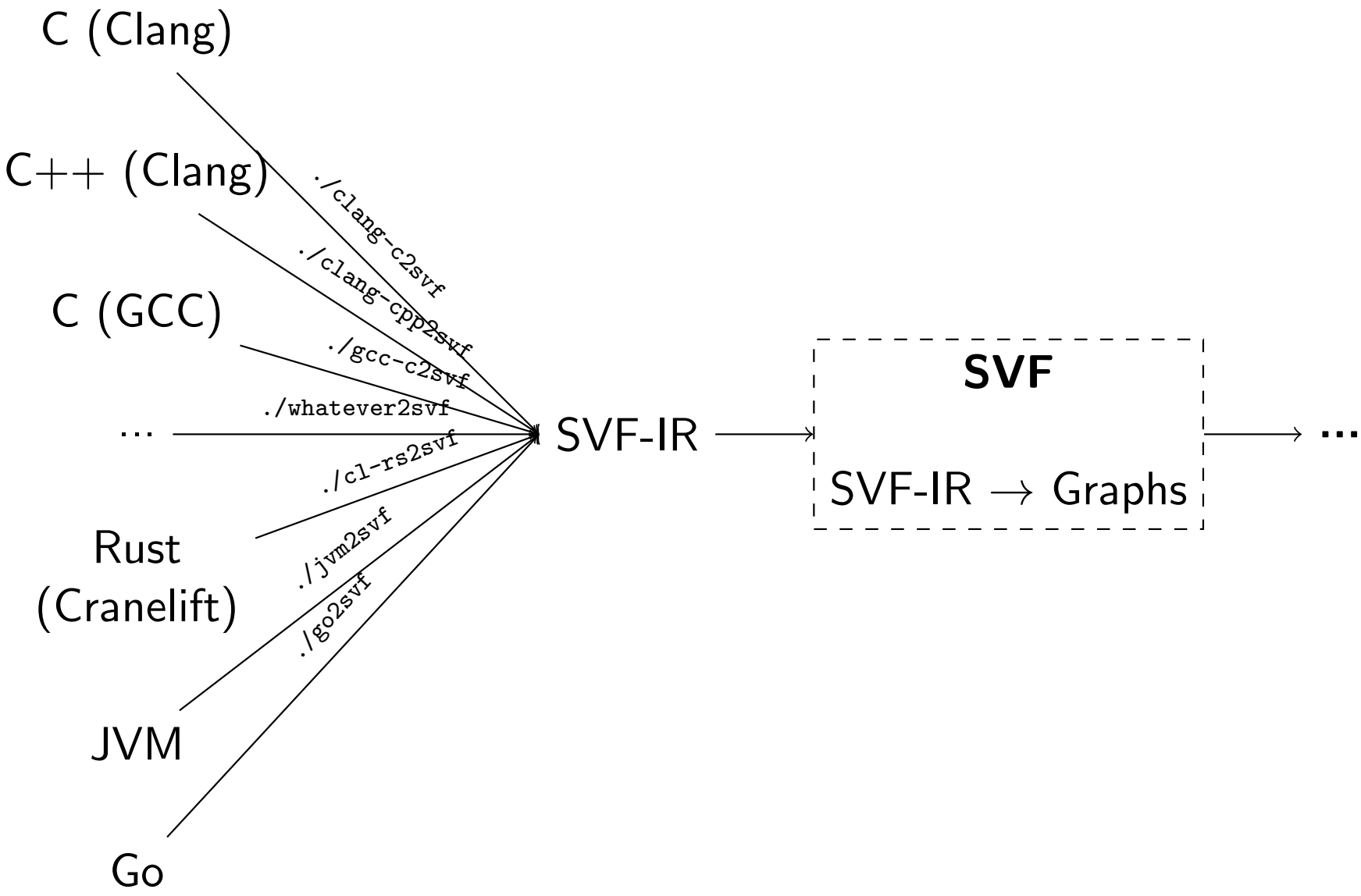
As it stands now...



Goal

Single, unifying input IR to SVF...

Goal


C (Clang)
C++ (Clang)
C (GCC)
...
Rust
(Cranelift)
JVM
Go
./clang-c2svf
./clang-cpp2svf
./gcc-c2svf
./whatever2svf
./cl-rs2svf
./jvm2svf
./go2svf
SVF-IR
SVF
SVF-IR → Graphs
...

Goal

- IR can be translated to from a variety of other languages/IRs

Goal

- IR can be translated to from a variety of other languages/IRs
- IR definition only concerned with analysis

Goal

- IR can be translated to from a variety of other languages/IRs
- IR definition only concerned with analysis
- IR (somewhat) lightweight

Goal

- IR can be translated to from a variety of other languages/IRs
- IR definition only concerned with analysis
- IR (somewhat) lightweight
- IR definition and tooling live outside SVF

Goal

- IR can be translated to from a variety of other languages/IRs
- IR definition only concerned with analysis
- IR (somewhat) lightweight
- IR definition and tooling live outside SVF
- IR tooling depends on other language tooling but only optionally

Goal

- IR can be translated to from a variety of other languages/IRs
- IR definition only concerned with analysis
- IR (somewhat) lightweight
- IR definition and tooling live outside SVF
- IR tooling depends on other language tooling but only optionally
- Sufficiently expressive for external APIs

Goal

- IR can be translated to from a variety of other languages/IRs
- IR definition only concerned with analysis
- IR (somewhat) lightweight
- IR definition and tooling live outside SVF
- IR tooling depends on other language tooling but only optionally
- Sufficiently expressive for external APIs
- IR extensible with language-specific constructs

Non-goals

- IR is executable

Non-goals

- IR is executable
- IR is optimisable

Non-goals

- IR is executable
- IR is optimisable
- IR expressive enough for all languages

Non-goals

- IR is executable
- IR is optimisable
- IR expressive enough for all languages
- IR suited to non-SVF analyses

Example

```
struct node {
    int val;
    struct node *next;
};

const unsigned MAX_LENGTH = 10;

void dbg(void *);
unsigned count(const struct node *list) {
    unsigned count = 0;
    const struct node *curr = list;
    while (curr) {
        ++count;
        curr = curr->next;
    }
    dbg(&count);
    return count;
}
```

Example

```
(preamble
  (version 0.0)
  (source hand))
(types
  (~struct.node (agg 2)))
(variables
  (@MAX_LENGTH (10 i32)))
(functions
  (@dbg (%0 ptr) void opaque)
  (@count ((%0 ptr)) i32 (
    (!1 (
      (alloc stack %2 (md (type ptr)))
      (alloc stack %3 (md (type i32)))
      (alloc stack %4 (md (type ptr)))
      (store %0 %2)
      (store (0 i32) %3)
      (load (%5 ptr) %2)
      (store %5 %4)
      (br !2)))
    (!2 (
      (load (%7 ptr) %4)
      (cmp != %8 %7 (null ptr))
      (brif %8 !3 !4)))
    (!3 (
      (load (%10 i32) %3)
      (add %11 %10 (1 i32))
      (store %11 %3)
      (load (%12 ptr) %4)
      (field %13 %12 1)
      (load (%14 ptr) %13)
      (store %14 %4)
      (br !2)))
    (!4 (
      (load (%16 i32) %3)
      (call void @dbg (@count))
      (ret %16))))))
```

Identifiers

- Local IDs: `%foo`, `%bar.baz`, `%3`

Identifiers

- Local IDs: `%foo`, `%bar.baz`, `%3`
- Global IDs: `@x`, `@strlen`

Identifiers

- Local IDs: `%foo`, `%bar.baz`, `%3`
- Global IDs: `@x`, `@strlen`
- Block IDs: `!a`, `!2`

Identifiers

- Local IDs: `%foo`, `%bar.baz`, `%3`
- Global IDs: `@x`, `@strlen`
- Block IDs: `!a`, `!2`
- Type IDs: `~t`, `~struct.fred`

Types

- void

Types

- void
- ptr

Types

- void
- ptr
- integers: i8, i16, i32, i64

Types

- void
- ptr
- integers: i8, i16, i32, i64
- floats (IEEE 754): f16, f32, f64, f128

Types

- void
- ptr
- integers: i8, i16, i32, i64
- floats (IEEE 754): f16, f32, f64, f128
- aggregations: sequences of n fields

Types

- void
- ptr
- integers: i8, i16, i32, i64
- floats (IEEE 754): f16, f32, f64, f128
- aggregations: sequences of *n* fields
  - `(agg 15)`

Types

- void
- ptr
- integers: i8, i16, i32, i64
- floats (IEEE 754): f16, f32, f64, f128
- aggregations: sequences of *n* fields
  - `(agg 15)`
  - `(agg 0)`

Memory Instructions

```
| '(' 'alloc' ('heap'|'stack') <lid> <md>? ')'
-> (alloc heap %foo (md (size 64)))
-> (alloc stack %foo (md (type i32)))
```

Memory Instructions

```
| '(' 'alloc' ('heap'|'stack') <lid> <md>? ')'
-> (alloc heap %foo (md (size 64)))
-> (alloc stack %foo (md (type i32)))


| '(' 'store' <val> <pval> <md>? ')'
-> (store (5 i32) %2)
```

Memory Instructions

```
| '(' 'alloc' ('heap'|'stack') <lid> <md>? ')'
-> (alloc heap %foo (md (size 64)))
-> (alloc stack %foo (md (type i32)))


| '(' 'store' <val> <pval> <md>? ')'
-> (store (5 i32) %2)


| '(' 'load' <typed-lid> <pval> <md>? ')'
-> (load (%x ptr) %y)
-> (load (%x ptr) null)
```

Memory Instructions

```
| '(' 'alloc' ('heap'|'stack') <lid> <md>? ')'
-> (alloc heap %foo (md (size 64)))
-> (alloc stack %foo (md (type i32)))


| '(' 'store' <val> <pval> <md>? ')'
-> (store (5 i32) %2)


| '(' 'load' <typed-lid> <pval> <md>? ')'
-> (load (%x ptr) %y)
-> (load (%x ptr) null)


| '(' 'field' <lid> <pval> <ival> <md>? ')'
-> (field %f %o 3)
-> (field %f %o %i)
```

Arithmetic Instructions

```
| '(' 'add' <lid> <val> <val> <md>? ')'
-> (add %sum %x %y)
```

Arithmetic Instructions

```
| '(' 'add' <lid> <val> <val> <md>? ')'
-> (add %sum %x %y)


| '(' 'sub' <lid> <val> <val> <md>? ')'
-> (sub %x (10 i32) (5 i32))
```

Arithmetic Instructions

```
| '(' 'add' <lid> <val> <val> <md>? ')'
-> (add %sum %x %y)


| '(' 'sub' <lid> <val> <val> <md>? ')'
-> (sub %x (10 i32) (5 i32))


| '(' 'mul' <lid> <val> <val> <md>? ')'
-> (mul %prod (10 i64) %x)
```

Arithmetic Instructions

```
| '(' 'add' <lid> <val> <val> <md>? ')'
-> (add %sum %x %y)


| '(' 'sub' <lid> <val> <val> <md>? ')'
-> (sub %x (10 i32) (5 i32))


| '(' 'mul' <lid> <val> <val> <md>? ')'
-> (mul %prod (10 i64) %x)


| '(' 'div' ('unsigned'|'signed') <lid> <val> <val> <md>? ')'
-> (div signed %d %x (i32 5))
```

Arithmetic Instructions

```
| '(' 'add' <lid> <val> <val> <md>? ')'
-> (add %sum %x %y)


| '(' 'sub' <lid> <val> <val> <md>? ')'
-> (sub %x (10 i32) (5 i32))


| '(' 'mul' <lid> <val> <val> <md>? ')'
-> (mul %prod (10 i64) %x)


| '(' 'div' ('unsigned'|'signed') <lid> <val> <val> <md>? ')'
-> (div signed %d %x (i32 5))


| '(' 'rem' ('unsigned'|'signed') <lid> <val> <val> <md>? ')'
-> (rem unsigned %r %foo %bar)
```

Bitwise Instructions

```
| '(' 'not' <lid> <val> <md>? ')'
-> (not %r (0 i8))
```

Bitwise Instructions

```
| '(' 'not' <lid> <val> <md>? ')'
-> (not %r (0 i8))


| '(' 'and' <lid> <val> <val> <md>? ')'
-> (and %r (8 i8) %x)
```

Bitwise Instructions

```
| '(' 'not' <lid> <val> <md>? ')'
-> (not %r (0 i8))

| '(' 'and' <lid> <val> <val> <md>? ')'
-> (and %r (8 i8) %x)

| '(' 'or' <lid> <val> <val> <md>? ')'
-> (or %r (1 i16) %x)
```

Bitwise Instructions

```
| '(' 'not' <lid> <val> <md>? ')'
-> (not %r (0 i8))

| '(' 'and' <lid> <val> <val> <md>? ')'
-> (and %r (8 i8) %x)

| '(' 'or' <lid> <val> <val> <md>? ')'
-> (or %r (1 i16) %x)

| '(' 'xor' <lid> <val> <val> <md>? ')'
-> (xor %r %a %x)
```

Bitwise Instructions

```
| '(' 'not' <lid> <val> <md>? ')'
-> (not %r (0 i8))

| '(' 'and' <lid> <val> <val> <md>? ')'
-> (and %r (8 i8) %x)

| '(' 'or' <lid> <val> <val> <md>? ')'
-> (or %r (1 i16) %x)

| '(' 'xor' <lid> <val> <val> <md>? ')'
-> (xor %r %a %x)

| '(' 'shiftl' <lid> <val> <val> <md>? ')'
-> (shiftl %r %a (4 i32))
```

Bitwise Instructions

```
| '(' 'not' <lid> <val> <md>? ')'
-> (not %r (0 i8))


| '(' 'and' <lid> <val> <val> <md>? ')'
-> (and %r (8 i8) %x)


| '(' 'or' <lid> <val> <val> <md>? ')'
-> (or %r (1 i16) %x)


| '(' 'xor' <lid> <val> <val> <md>? ')'
-> (xor %r %a %x)


| '(' 'shiftl' <lid> <val> <val> <md>? ')'
-> (shiftl %r %a (4 i32))


| '(' 'shiftr' ('logical'|'arithmetic') <lid> <val> <val> <md>? ')'
-> (shiftr logical %z %x %y)
```

Control Instructions

```
| '(' 'phi' <lid> '(' ('(' <val> <bid> ')')+ ')' <md>? ')'
-> (phi %p (%x !2) (%y !3) (%z !z))
-> (phi %p (%x !2))
```

Control Instructions

```
| '(' 'phi' <lid> '(' ('(' <val> <bid> ')')+ ')' <md>? ')'
-> (phi %p (%x !2) (%y !3) (%z !z))
-> (phi %p (%x !2))

| '(' 'call' ('void'|<lid>) <pval> '(' <val>* ')' <md>? ')'
-> (call %r @fun (%x (4 i32)))
-> (call void @donothing ())
```

Control Instructions

```
| '(' 'phi' <lid> '(' ('(' <val> <bid> ')')+ ')' <md>? ')'
-> (phi %p (%x !2) (%y !3) (%z !z))
-> (phi %p (%x !2))

| '(' 'call' ('void'|<lid>) <pval> '(' <val>* ')' <md>? ')'
-> (call %r @fun (%x (4 i32)))
-> (call void @donothing ())

| '(' 'br' <bid> <md>? ')'
-> (br !blk)
```

Control Instructions

```
| '(' 'phi' <lid> '(' ('(' <val> <bid> ')')+ ')' <md>? ')'
-> (phi %p (%x !2) (%y !3) (%z !z))
-> (phi %p (%x !2))

| '(' 'call' ('void'|<lid>) <pval> '(' <val>* ')' <md>? ')'
-> (call %r @fun (%x (4 i32)))
-> (call void @donothing ())

| '(' 'br' <bid> <md>? ')'
-> (br !blk)

| '(' 'brif' <val> <bid> <bid> <md>? ')'
-> (brif %c !2 !5)
```

Control Instructions

```
| '(' 'phi' <lid> '(' ('(' <val> <bid> ')')+ ')' <md>? ')'
-> (phi %p (%x !2) (%y !3) (%z !z))
-> (phi %p (%x !2))

| '(' 'call' ('void'|<lid>) <pval> '(' <val>* ')' <md>? ')'
-> (call %r @fun (%x (4 i32)))
-> (call void @donothing ())

| '(' 'br' <bid> <md>? ')'
-> (br !blk)

| '(' 'brif' <val> <bid> <bid> <md>? ')'
-> (brif %c !2 !5)

| '(' 'ret' (<val>|'void') <md>? ')'
-> (ret %v)
-> (ret (null ptr))
-> (ret void)
```

Comparison Instruction

```
| '(' 'cmp' <cmp-op> <lid> <val> <val> <md>? ')'
-> (cmp = %foo %x %y)
-> (cmp > %bar (5 i32) (4 i32))
-> (cmp != %baz %v (4 i32))
```

Conversion Instructions

```
| '(' 'reinterpret' <lid> <val> <type> <md>? ')'
-> (reinterpret %newf (5 i64) f64)
-> (reinterpret %newi (5 f32) i32)
```

Conversion Instructions

```
| '(' 'reinterpret' <lid> <val> <type> <md>? ')'
-> (reinterpret %newf (5 i64) f64)
-> (reinterpret %newi (5 f32) i32)


| '(' 'convert' <lid> <val> <type> <md>? ')'
-> (convert %r (5 f32) i64)
-> (convert %r (5 i32) f64)
-> (convert %r (null ptr) i64)
-> (convert %r %z ptr2)
```

Conversion Instructions

```
| '(' 'reinterpret' <lid> <val> <type> <md>? ')'
-> (reinterpret %newf (5 i64) f64)
-> (reinterpret %newi (5 f32) i32)

| '(' 'convert' <lid> <val> <type> <md>? ')'
-> (convert %r (5 f32) i64)
-> (convert %r (5 i32) f64)
-> (convert %r (null ptr) i64)
-> (convert %r %z ptr2)

| '(' 'inttrunc' <lid> <val> <type> <md>? ')'
-> (inttrunc %smaller (100 i32) i8)
```

Conversion Instructions

```
| '(' 'reinterpret' <lid> <val> <type> <md>? ')'
-> (reinterpret %newf (5 i64) f64)
-> (reinterpret %newi (5 f32) i32)


| '(' 'convert' <lid> <val> <type> <md>? ')'
-> (convert %r (5 f32) i64)
-> (convert %r (5 i32) f64)
-> (convert %r (null ptr) i64)
-> (convert %r %z ptr2)


| '(' 'inttrunc' <lid> <val> <type> <md>? ')'
-> (inttrunc %smaller (100 i32) i8)


| '(' 'intext' ('zero'|'sign') <lid> <val> <type> <md>? ')'
-> (intext zero %bigger (100 i32) i64)
-> (intext sign %bigger (-3 i8) i16)
```

Other Instructions

```
| '(' 'assign' <lid> <val> <md>? ')'
-> (assign %x (+inf f64))
-> (assign %y (5.5e4 f32))
```

Other Instructions

```
| '(' 'assign' <lid> <val> <md>? ')'
-> (assign %x (+inf f64))
-> (assign %y (5.5e4 f32))

| '(' 'vararg' <typed-lid> <ival> <md>? ')'
-> (vararg (%p ptr) 3)
-> (vararg (%v i64) %i)
```

Other Instructions

```
| '(' 'assign' <lid> <val> <md>? ')'
-> (assign %x (+inf f64))
-> (assign %y (5.5e4 f32))
```

```
| '(' 'vararg' <typed-lid> <ival> <md>? ')'
-> (vararg (%p ptr) 3)
-> (vararg (%v i64) %i)
```

```
| '(' 'blackhole' ('contained'|'uncontained') <typed-lid> <md>? ')'
-> (blackhole uncontained (%p ptr) (md (asm "movl %eax, %ebx\n...")))
-> (blackhole contained (%i i32))
```

The Preamble

```
(preamble
  (version 0.0)
  (source hand))
```

The Preamble

```
(preamble
  (version 0.0)
  (source hand))

(preamble
  (version 2.1)
  (source clangcpp))
```

Extensibility

Our IR cannot be expressive enough for all languages without making a mess.

Extensibility

Our IR cannot be expressive enough for all languages without making a mess.

1. Version. Features can be added and removed and the consumer of the IR can understand what to expect.

Extensibility

Our IR cannot be expressive enough for all languages without making a mess.

1. Version. Features can be added and removed and the consumer of the IR can understand what to expect.
2. Metadata. For example, vtables can be given as metadata for to analyse C++ programs more efficiently and precisely. To be understood alongside given `source`.

Practicalities

SVF-IR repository at `github.com/SVF-Tools/SVF-IR`

Practicalities

SVF-IR repository at `github.com/SVF-Tools/SVF-IR`

Practicalities

SVF-IR repository at `github.com/SVF-Tools/SVF-IR`

- `svfirlib`: AST, parser, and other tools to operate on SVF-IR
  - No dependencies
  - SVF is to depend on this library *only*

Practicalities

SVF-IR repository at `github.com/SVF-Tools/SVF-IR`

- `svfirlib`: AST, parser, and other tools to operate on SVF-IR
  - No dependencies
  - SVF is to depend on this library *only*
- Translation tools: standalone programs to translate other languages to SVF-IR.
  - `clangcpp2svf`, `llvm2svf`, `jvm2svf`, etc.
  - Not required to use SVF-IR
  - Orthogonal to each other (e.g., `jvm2svf` does not require LLVM like `llvm2svf`)

Practicalities

What is done?

Practicalities

What is done?

- The language, for the most part

Practicalities

What is done?

- The language, for the most part
- Grammar for text format

Practicalities

What is done?

- The language, for the most part
- Grammar for text format
- Parser for said text format

Practicalities

What is done?

- The language, for the most part
- Grammar for text format
- Parser for said text format
- In-memory AST

Practicalities

What is done?

- The language, for the most part
- Grammar for text format
- Parser for said text format
- In-memory AST
- Type-checking/typer in the works

Practicalities

What is done?

- The language, for the most part
- Grammar for text format
- Parser for said text format
- In-memory AST
- Type-checking/typer in the works
- Some debugging tools

Practicalities

What is to be done?

Practicalities

What is to be done?

- Spec (prose) for the language

Practicalities

What is to be done?

- Spec (prose) for the language
- Serialised/binary format

Practicalities

What is to be done?

- Spec (prose) for the language
- Serialised/binary format
- Have SVF consume SVF-IR

Practicalities

What is to be done?

- Spec (prose) for the language
- Serialised/binary format
- Have SVF consume SVF-IR
- `llvm2svf`

Practicalities

What is to be done?

- Spec (prose) for the language
- Serialised/binary format
- Have SVF consume SVF-IR
- `llvm2svf`
  - Pulled from the current SVF→graphs translator

Practicalities

What is to be done?

- Spec (prose) for the language
- Serialised/binary format
- Have SVF consume SVF-IR
- `llvm2svf`
  - Pulled from the current SVF→graphs translator
  - Port external APIs from C to new IR

Practicalities

What is to be done?

- Spec (prose) for the language
- Serialised/binary format
- Have SVF consume SVF-IR
- `llvm2svf`
  - Pulled from the current SVF→graphs translator
  - Port external APIs from C to new IR
  - Virtual tables as metadata

Practicalities

What is to be done?

- Spec (prose) for the language
- Serialised/binary format
- Have SVF consume SVF-IR
- `llvm2svf`
  - Pulled from the current SVF→graphs translator
  - Port external APIs from C to new IR
  - Virtual tables as metadata
- TESTS

That's all, thanks

github.com/SVF-Tools/SVF-IR